A
Major Project Report
On

MULTILEVEL DATA CONCEALING USING STEGANOGRAPHY AND VISUAL CRYPTOGRAPHY

Submitted in partial fulfillment of the requirements for the award of degree

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted By

K.VENKATARAMANA 227Z1A0578

Under the Guidance of

Mr. K. Anjaiah
Assistant Professor

SCHOOL OF ENGINEERING
Department of Computer Science and Engineering

NALLA NARASIMHA REDDY
EDUCATION SOCIETY'S GROUP OF INSTITUTIONS
(AN AUTONOMOUS INSTITUTION)

Approved by AICTE, New Delhi, Chowdariguda (V) Korremula 'x' Roads, via Narapally, Ghatkesar (Mandal) Medchal (Dist), Telangana-500088

2025-2026

SCHOOL OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report titled **"MULTILEVEL DATA CONCEALING USING STEGANOGRAPHY AND VISUAL CRYPTOGRAPHY"** is being submitted by **K.Venkataramana (227Z1A0578)** in Partial fulfillment for the award of **Bachelor of Technology in Computer Science and Engineering** is a record bonafide work carried out by him. The results embodied in this report have not been submitted to any other university for the award of any degree.

Internal Guide
(Mr. K. Anjaiah)

Head of the Department
(Dr.K.Rameshwaraiah)

Submitted for Viva voce Examination held on…….

External Examiner

DECLARATION

I, K. Venkataramana the student of **Bachelor of Technology in Computer Science and Engineering, Nalla Narasimha Reddy Education Society's Group of Institutions**, Hyderabad, Telangana, hereby declare that the work presented in this project work entitled **MULTILEVEL DATA CONCEALING USING STEGANOGRAPHY AND VISUAL CRYPTOGRAPHY** is the outcome of my own bonafide work and is correct to the best of my knowledge and this work has been undertaken by taking care of engineering ethics. It contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning.

K.Venkataramana 227Z1A0578

Date:

Signature:

ACKNOWLEDGEMENT

I express my sincere gratitude to my guide **Mr. K. Anjaiah**, Assistant Professor, in Computer Science and Engineering Department, NNRESGI, who motivated throughout the period of the project and also for his valuable and intellectual suggestions apart from his adequate guidance, constant encouragement right throughout my work.

I wish to record my deep sense of gratitude to my Project coordinator **Mrs. Ch. Ramya**, Assistant Professor, in Computer Science and Engineering Department, NNRESGI, for offering valuable insights and advice during the review sessions, providing consistent guidance throughout the project, and giving us the opportunity to present my work.

I profoundly express my sincere thanks to **Dr. K. Rameshwaraiah**, Professor & Head, Department of Computer Science and Engineering, NNRESGI, has been of great help in carrying out the project work and is acknowledged with reverential thanks.

I wish to express my sincere thanks to **Dr. G. Janardhana Raju**, Dean School of Engineering, NNRESGI, for providing the facilities for completion of the project.

I am highly grateful to the **Dr. C. V. Krishna Reddy**, Director NNRESGI for providing the facilities for completion of the project.

I would like to thank Project Review Committee (PRC) members, all the faculty members and supporting staff, Department of Computer Science and Engineering, NNRESGI for their intellectual support throughout the course of this work.

Finally, I am indebted to all whosoever have contributed in this report work.

By:

K. Venkataramana 227Z1A0578

ABSTRACT

The project “Multilevel Data Concealing Using Steganography and Visual Cryptography” focuses on securing sensitive information through a two-layer protection system. In the first layer, steganography is used to hide secret data inside a cover medium such as an image, making the presence of information invisible to attackers. In the second layer, the concealed data is protected using visual cryptography, where the hidden content is divided into multiple encrypted shares; each share alone reveals nothing, and only the correct combination of shares can reconstruct the original secret. This hybrid approach ensures high confidentiality and robustness, as even if steganography is compromised, the data remains secure in cryptographic shares. The system supports multi-format concealment, n-out-of-n share generation, password-protected access, resistance to noise and compression attacks, and quality evaluation using metrics like PSNR and SSIM. It has practical applications in defence communication, medical data security, banking authentication, and e-governance, offering an invisible, unbreakable, and practical solution for secure data transmission.

Keywords: Steganography, Visual Cryptography, Data Concealment, Two-layer Security, Encrypted Shares, Confidentiality and Robustness, PSNR(Peak Signal-to-Noise Ratio), SSIM (Structural Similarity Index Measure), Secure Data Transmission

TABLE OF CONTENTS

	PageNo.
Abstract	
List of Figures	**i**
List of Tables	**ii**
List of Abbreviations	**iii**
1. INTRODUCTION	**1**
1.1 Motivation	2
1.2 Problem Definition	3
1.3 Objective of project	3
1.4 Limitation of project	4
2. LITERATURE SURVEY	**6**
2.1 Introduction	6
2.2 Existing System	8
2.3 Proposed Systemys	10
3. SYSTEM ANALYSIS	**12**
3.1 Functional requirements	12
3.2 Non-Functional requirements	12
3.3 Software requirements	13
3.4 Hardware requirements	13
3.5 Block Diagram of Proposed System	13
4. SYSTEM DESIGN	**15**
4.1 Introduction	15
4.2 UML Diagrams	16
4.3 Modules	21
5. IMPLEMENTATION	**23**
5.1 Introduction	23
5.2 Method of Implementation	24
5.3 Algorithm and Flowcharts	25
5.4 Control Flow of the implementation	27
5.5 Sample Code	29

6. TESTING **36**

6.1 Introduction 36

6.2 Types of Tests considered 37

6.3 Test case scenarios 38

7. OUTPUT SCREENS **41**

8. CONCLUSION AND FUTURE ENHANCEMENT **47**

8.1 Project Conclusion 47

8.2 Future Enhancement 48

9. REFERENCES **49**

9.1 Journals 49

9.2 Books 49

9.3 Sites 50

LIST OF FIGURES

S.No.	Figure No.	Name of the Figure	Page No.
1	2.2	Existing System	9
2	3.5	Block diagram of system	14
3	4.2.1	Class diagram	17
4	4.2.2	Activity diagram	18
5	4.2.3	Use case diagram	18
6	4.2.4	Sequence diagram	19
7	5.2	Agile method	25
8	5.4.1	Importing Dependencies	29
9	7.1	Output screen for Login and Registration	41
10	7.2	Output screen for Encoding	42
11	7.3	Output screen for Decoding	43
12	7.4	Output screen for Visual Cryptography	44
13	7.5	Output screen for Quality metrics	45
14	7.6	Output screen for Quality metrics result	46

LIST OF TABLES

S.No.	Table No.	Name of the Table	Page No.
1	6.3	Test cases for proposed system	39

LIST OF ABBREVIATIONS

S.No.	Abbreviation	Definitions
1	AES	Advanced Encryption Standard
2	API	Application Programming Interface
3	DES	Data Encryption Standard
4	LSB	Least Significant Bit
5	OCR	Optical Character Recognition
6	PSNR	Peak Signal-to-Noise Ratio
7	RAM	Random Access Memory
8	SSIM	Structural Similarity Index Measure
9	UI	User Interface
10	VC	Visual Cryptography

1. INTRODUCTION

The rapid growth of digital communication and information technology has significantly increased the demand for secure data transmission and storage. In today's digital era, sensitive information such as personal data, financial records, medical details, and confidential communications is frequently shared over networks. This makes data highly vulnerable to cyber threats, hacking, and unauthorized access. Although traditional encryption techniques provide a certain level of security, encrypted data can still attract attackers' attention. Therefore, there is a growing need for advanced security mechanisms that not only protect data but also hide its very existence from potential intruders.

To address this challenge, this project proposes a multilevel data concealing system that integrates steganography and visual cryptography for enhanced security. Steganography is used to embed secret information within a cover image in such a way that the modification is imperceptible to the human eye. This ensures that the presence of hidden data remains undetectable. Further, the stego image is processed using visual cryptography, where it is divided into multiple shares. Individually, these shares do not reveal any meaningful information, but when combined, they reconstruct the original hidden content. This dual-layer approach provides stronger protection compared to conventional single-layer security techniques.

In recent years, advancements in image processing, cyber security, and data hiding techniques have enabled the development of more efficient and robust systems. Steganography techniques such as Least Significant Bit (LSB) insertion allow data to be hidden within pixel values without significantly affecting image quality. On the other hand, visual cryptography ensures secure data sharing by splitting images into multiple parts, eliminating the need for complex decryption algorithms. The combination of these two techniques enhances both confidentiality and security, making the system highly reliable.

The proposed system is designed and implemented using Python and Streamlit, along with powerful libraries such as OpenCV, NumPy, and PIL for image processing operations. The system provides an interactive and user-friendly interface that allows users to easily encode secret text or images, generate cryptographic shares, and decode the concealed data when required. Additionally, performance evaluation metrics such as Peak Signal-to-Noise Ratio

(PSNR) and Structural Similarity Index Measure (SSIM) are used to analyze the quality of the stego images and ensure minimal distortion.

Furthermore, the system emphasizes data integrity and secure transmission. Even if an unauthorized user gains access to one of the shares or the stego image, it is nearly impossible to retrieve the original information without the proper combination of shares and decoding process. This makes the system suitable for applications such as secure communication, military data transfer, confidential document sharing, and digital watermarking.

Overall, the multilevel data concealing system demonstrates how the integration of steganography and visual cryptography can provide a powerful, secure, and efficient solution for protecting sensitive information. By combining data hiding with data splitting techniques, the system ensures both invisibility and robustness, thereby significantly reducing the risk of data breaches and enhancing secure communication in modern digital environments.

1.1 Motivation

In today's digital era, the rapid growth of internet usage and communication technologies has made data sharing faster and more convenient. However, this has also increased the risk of cyber attacks, data breaches, and unauthorized access to sensitive information. Personal, financial, and confidential data are constantly transmitted over networks, making security a major concern. Protecting this information from potential threats has become essential to ensure privacy and trust in digital communication systems.

Traditional security methods such as encryption are commonly used to safeguard data. Although encryption converts information into an unreadable format, it still reveals the presence of sensitive data, which may attract attackers. This limitation creates the need for more advanced techniques that not only protect data but also conceal its existence, reducing the chances of detection and attack.

This project is motivated by the idea of combining steganography and visual cryptography to create a multilevel security system. Steganography hides secret data within images, making it invisible to the human eye, while visual cryptography splits the concealed data into multiple shares that reveal no information individually. By integrating these two techniques, the system provides enhanced security, ensuring that even if one layer is compromised, the data remains protected.

Furthermore, the increasing demand for secure communication in fields such as banking, healthcare, and defense highlights the importance of reliable data protection methods. This project aims to address these challenges by developing a system that ensures confidentiality, integrity, and secure sharing of information. The motivation is to build a practical, efficient, and user-friendly solution that demonstrates the effectiveness of multilevel data concealing in modern cyber security applications.

1.2 Problem Definition

In the current digital environment, the secure transmission and storage of sensitive information have become a major challenge due to the increasing number of cyber threats, hacking attempts, and unauthorized access. Traditional data protection methods such as encryption provide security by converting data into an unreadable format, but they do not hide the presence of the data itself. This makes encrypted information a potential target for attackers, increasing the risk of data breaches.

Additionally, existing single-layer security techniques are often not sufficient to provide complete protection. If the encryption is broken or the hidden data is detected, the entire information can be compromised. There is also a lack of efficient systems that combine multiple security mechanisms to enhance both data confidentiality and invisibility while maintaining ease of use.

Therefore, there is a need for a robust and reliable system that can provide multilevel security by not only protecting the content of the data but also concealing its existence. The system should ensure that even if one layer of security is compromised, the data remains secure and inaccessible without proper authorization.

This project aims to address these challenges by developing a multilevel data concealing system using steganography and visual cryptography. The system focuses on securely embedding secret data within images and further splitting it into multiple shares, thereby enhancing security, preventing unauthorized access, and ensuring safe communication of sensitive information.

1.3 Objective of Project

The main objective of this project is to develop a secure multilevel data concealing system that protects sensitive information by combining steganography and visual cryptography

techniques. The system aims to enhance data security by hiding the existence of information and ensuring that it can only be accessed through proper reconstruction methods

The specific objectives of the project are as follows:

- To develop a system for hiding secret data within digital images using steganography.
- To implement Least Significant Bit (LSB) technique for efficient and invisible data embedding.
- To apply visual cryptography to divide the stego image into multiple secure shares.
- To ensure that the original data can only be reconstructed by combining the correct shares.
- To design a user-friendly interface using Streamlit for encoding and decoding processes.
- To evaluate the quality and performance of the system using metrics like PSNR and SSIM.
- To develop a reliable solution for secure data transmission, storage, and confidential communication.

1.4 Limitations of Project

The proposed multilevel data concealing system has some limitations despite providing enhanced security. The effectiveness of steganography depends on image quality, and hiding large data may cause distortion, while visual cryptography requires all shares for reconstruction, making data recovery impossible if any share is lost. The amount of data that can be hidden depends on the size of the image. Additionally, the system is mainly designed for images and may not support other formats without changes.Additionally, the system may have higher processing time for large data and is mainly limited to image-based concealment.

1.4.1 Processing Time

The system may take more time to hide and retrieve data because it uses two security techniques—steganography and visual cryptography. In the first step, the secret data is embedded into an image, and in the second step, it is divided into multiple encrypted shares. During retrieval, these steps are reversed, which also requires time.Since the data passes through multiple stages such as embedding, share generation, and extraction, the overall process becomes slower compared to single-method systems. Additionally, larger image sizes or more hidden data can further increase processing time, affecting system performance on low-end devices.

1.4.2 Language Dependent

The system may be limited in handling different languages if text data is used for hiding information. It may not properly support all languages or special characters. This can affect accuracy and usability for users with different language requirements.

1.4.3 Noisy Environment

The system performance may be affected if the input data (like images) is corrupted by noise during transmission. Noise can distort the hidden data, making it difficult to extract the original information correct.

2. LITERATURE SURVEY

2.1 Introduction

The literature survey provides an overview of existing research and techniques related to data security, steganography, and visual cryptography. With the increasing need for secure communication, various methods have been developed to protect sensitive information from unauthorized access. Researchers have explored different approaches such as encryption, data hiding, and image processing techniques to enhance data confidentiality and integrity.

Many existing systems focus on single-layer security methods like cryptography or steganography. While these techniques provide a certain level of protection, they have limitations such as visibility of encrypted data, vulnerability to attacks, or data loss during processing. To overcome these challenges, recent studies have focused on combining multiple techniques to achieve stronger and more reliable security.

This literature survey analyzes different research works that use methods like AES encryption, LSB-based steganography, transform-based techniques, and visual cryptography. It highlights their methodologies, advantages, and limitations, providing a clear understanding of current advancements in the field.

The survey also helps in identifying the research gap, which leads to the development of the proposed multilevel data concealing system. By combining steganography and visual cryptography, the project aims to overcome the limitations of existing methods and provide a more secure and efficient solution for data protection.

Traditional security systems mainly rely on encryption algorithms to protect data. In the work by M. Senthil Murugan, Udaya Kumar S, and Sooraj Mano Ukram T G (2025), an image encryption system based on the AES algorithm and Role-Based Access Control (RBAC) is proposed. This approach ensures strong confidentiality and controlled access to data, making it suitable for enterprise-level applications. However, it focuses only on encryption and does not hide the existence of data, which may still attract attackers and increase the risk of targeted attacks.

To overcome the limitation of visibility in encryption, steganography techniques have been widely used. In the study by Nishita N Murthy and Vinay Hegde (2024), a secure data hiding framework using Least Significant Bit (LSB) steganography combined with AES encryption is presented. This method provides multi-layer security by embedding encrypted

data into images. Although it improves confidentiality, the LSB technique is highly sensitive to image compression, noise, and other modifications, which may lead to loss or corruption of hidden data.

Further advancements in steganography have been made using transform-based and deep learning approaches. In the research by Brindha Subburaj, Aditya Sai, and Shantanu (2025), a multilevel Discrete Cosine Transform (DCT) combined with Convolutional Neural Networks (CNN) is used for secure image steganography. This approach improves robustness and maintains high image quality while hiding data. However, the system requires high computational power and complex implementation, which may not be suitable for lightweight or real-time applications.

Another significant contribution is presented by V. Sudharshan, Vineeth Surapuneni, and Sameer Shaik (2023), where secure multi-party computation is combined with reversible watermarking in encrypted images using AES. This method ensures secure data sharing and ownership verification. While it enhances security, the approach involves high complexity and is mainly focused on watermarking and authentication rather than general-purpose data concealment.

In addition to steganography and encryption, visual cryptography plays an important role in secure data sharing. The concept introduced by Moni Naor and Adi Shamir divides an image into multiple shares such that each share individually does not reveal any information. Only when the required shares are combined, the original image can be reconstructed. This method provides strong security and eliminates the need for complex decryption algorithms. However, it lacks integration with data hiding techniques, limiting its effectiveness when used alone.

Recent research trends focus on combining multiple techniques to achieve stronger and more reliable security. Hybrid approaches that integrate steganography with cryptography or visual cryptography have shown improved performance in terms of confidentiality, robustness, and resistance to attacks. However, many existing systems either suffer from high computational complexity, limited scalability, or lack of user-friendly implementation.

Overall, the analysis of existing literature reveals that most methods provide either data hiding or encryption or secure sharing, but not a complete multilevel security solution. There is a clear research gap in developing a system that combines these techniques effectively while maintaining efficiency and usability. The proposed project addresses this gap by integrating

steganography and visual cryptography into a unified framework. This ensures that the data is not only hidden but also securely divided into shares, providing enhanced protection against unauthorized access and cyber threats.

2.2 Existing System

In the current digital environment, several methods are used to secure sensitive data during transmission and storage. The most commonly used approach is cryptography, where data is encrypted into an unreadable format using algorithms such as AES (Advanced Encryption Standard), DES, and RSA. These techniques ensure that only authorized users with the correct decryption key can access the original data. While cryptography provides strong security, it has a major limitation: it does not hide the presence of the data. Encrypted data can easily attract attackers, making it a target for cryptanalysis and brute-force attacks.

Another widely used technique is steganography, which focuses on hiding secret data within digital media such as images, audio, or video files. Among various methods, the Least Significant Bit (LSB) technique is commonly used to embed data into image pixels without causing noticeable changes. This method ensures that the hidden data is invisible to the human eye. However, steganography alone is not completely secure. If an attacker suspects the presence of hidden data and extracts it, the entire information can be compromised. Additionally, steganographic methods are sensitive to image processing operations like compression, resizing, and noise, which may lead to data loss.

Visual cryptography is also used as a security technique, where an image is divided into multiple shares. Each share appears as random noise and does not reveal any meaningful information individually. Only when the required shares are combined, the original image is reconstructed. This method provides strong security and does not require complex decryption algorithms. However, visual cryptography alone does not hide data within a cover medium, and it is mainly used for secure image sharing rather than data concealment.

Most existing systems rely on a single-layer security approach, such as only encryption or only data hiding. These approaches have certain drawbacks, including lack of invisibility, vulnerability to attacks, and risk of data exposure if the method is compromised. Additionally, some advanced techniques like transform-based steganography and deep learning models improve robustness and accuracy but require high computational resources and complex implementation, making them less practical for general use.

Furthermore, existing systems often lack integration between different security techniques, which limits their overall effectiveness. They may also face challenges such as reduced efficiency, increased processing time for large data, and limited support for multiple media types. As a result, there is a need for a more comprehensive and secure solution that overcomes these limitations.

The existing systems provide basic security through encryption, data hiding, or sharing techniques, they fail to offer a complete and robust multilevel protection mechanism. These limitations highlight the need for a system that combines multiple security approaches to ensure better confidentiality, data protection, and resistance against modern cyber threats.

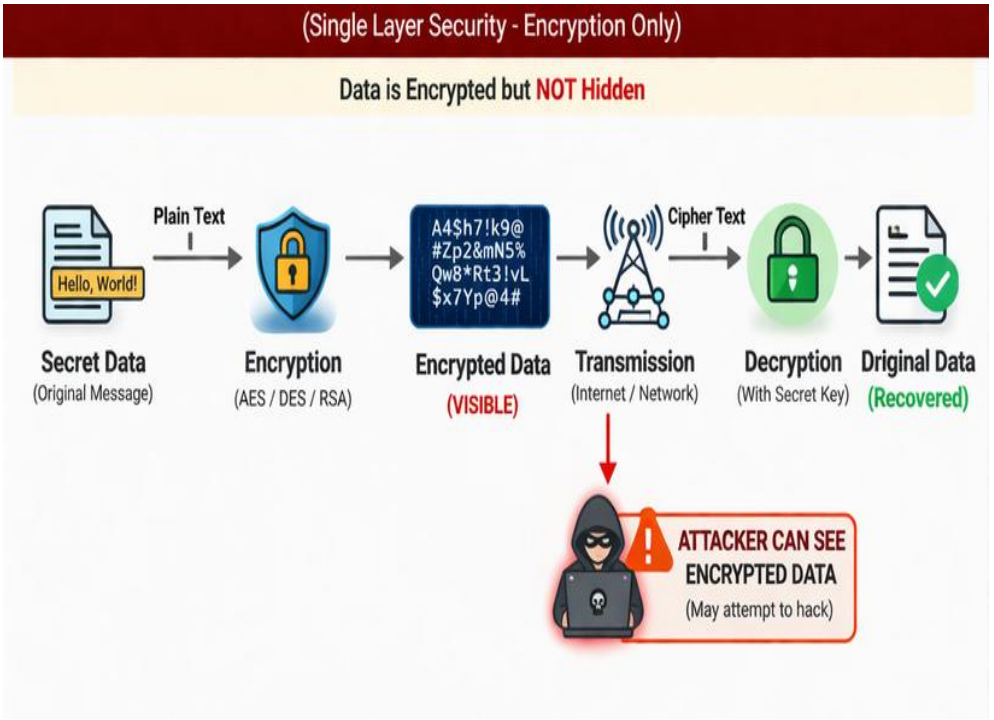

Fig 2.2: Existing system

2.2.1 Drawbacks of Existing System

- Provides only single-layer security (only encryption).
- Does not hide data, making it visible to attackers.
- More vulnerable to brute-force and cryptanalysis attacks.
- If encryption is broken, entire data is exposed.
- Lacks multilevel protection and advanced security features.

2.3 Proposed System

The proposed system introduces a multilevel data concealing approach that enhances the security of sensitive information by combining two powerful techniques: steganography and visual cryptography. Unlike traditional systems that rely on a single layer of security, this system ensures both data invisibility and data protection, making it highly secure against unauthorized access and cyber threats.

In this system, the process begins with the input of secret data, which can be in the form of text or an image. This data is first embedded into a cover image using a steganographic technique, specifically the Least Significant Bit (LSB) method. The LSB technique hides the secret information within the pixel values of the image in such a way that the changes are not noticeable to the human eye. As a result, the output is a stego image that appears identical to the original image but contains hidden data.

After embedding the data, the stego image undergoes a second level of security using visual cryptography. In this stage, the stego image is divided into multiple shares (for example, two or more shares). Each share looks like random noise and does not reveal any meaningful information on its own. These shares are then distributed or stored separately. Only when the correct shares are combined, the original stego image is reconstructed.

Once the shares are combined, the system performs the decoding process. The reconstructed image is processed to extract the hidden data using the reverse steganography technique. This ensures that only authorized users who possess the required shares and decoding mechanism can retrieve the original secret information. This two-step process significantly enhances data security, as breaking one layer does not compromise the entire system.

The system is implemented using Python and Streamlit, providing an interactive and user-friendly interface. Libraries such as OpenCV, NumPy, and PIL are used for image processing and data manipulation. The system allows users to perform operations such as encoding secret data, generating cryptographic shares, combining shares, and decoding hidden information through simple interface controls.

Additionally, the performance of the system is evaluated using quality metrics such as Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index Measure (SSIM). These metrics ensure that the stego image maintains high visual quality and that the hidden data does not significantly distort the original image. This is important for maintaining both security and usability.

The proposed system offers several advantages over existing systems. It provides multilevel security, ensures data confidentiality, and reduces the risk of detection by hiding the existence of data. Even if an attacker gains access to the stego image or one of the shares, it is extremely difficult to retrieve the original information without the complete set of shares and decoding process.

The proposed multilevel data concealing system provides a robust, efficient, and secure solution for protecting sensitive information. By integrating steganography and visual cryptography, the system ensures both invisibility and strong protection, making it a reliable approach for modern data security challenges.

2.3.1 Advantages of Proposed System

- Provides multilevel security using steganography and visual cryptography.
- Ensures data is hidden and protected from unauthorized access.
- Shares do not reveal information individually, increasing security
- Maintains good image quality with minimal distortion.
- Reduces risk of attacks by hiding data existence.

3. SYSTEM ANALYSIS

3.1 Functional Requirements

Functional requirements describe the specific functions and operations that the system must perform to achieve its objectives. The main functional requirements of the proposed system are as follows:

- The system should allow users to input secret data (text or image).
- The system should select and upload a cover image for data hiding.
- The system should embed secret data into the image using LSB steganography.
- The system should generate a stego image after embedding the data.
- The system should apply visual cryptography to split the stego image into multiple shares.
- The system should allow users to download and store generated shares.
- The system should combine shares to reconstruct the original stego image.
- The system should extract hidden data from the reconstructed image.
- The system should display output results such as images and extracted data.
- The system should evaluate performance using PSNR and SSIM metrics.

3.2 Non - Functional Requirements

Non-functional requirements define the quality attributes and performance characteristics of the system.

- The system should provide high security to protect sensitive data.
- The system should ensure good performance with reasonable processing time.
- The system should maintain high image quality after data embedding.
- The system should be user-friendly and easy to operate
- The system should be reliable in encoding and decoding data accurately.
- The system should ensure data integrity without loss during processing.
- The system should be scalable to handle different data sizes.
- The system should be compatible with common image formats (JPEG, PNG).
- The system should require low system resources for execution.
- The system should be maintainable and extensible for future improvements.

3.3 Software Requirements

The following software tools and technologies are used for the development of the tumble detection system:

1. Operating System: Windows / Linux
2. Web browser: Chrome, Microsoft Edge, Mozilla Firefox
3. Software: Visual Studio
4. Programming language: Python
5. Framework: Streamlit
6. Libraries: OpenCV, NumPy, PIL
7. Image Processing: OpenCV for image manipulation and steganography
8. Performance Metrics: PSNR, SSIM for image quality evaluation

3.4 Hardware Requirements

The hardware components required to run the system include:

1. Processor: Intel i5 or higher
2. RAM: Minimum 8 GB
3. Storage: sufficient space for image and data

3.5 Block Diagram of Proposed System

The block diagram of the proposed system represents the overall workflow of the multilevel data concealing process, which integrates steganography and visual cryptography to ensure enhanced security. The system is designed to perform both encoding (data hiding) and decoding (data retrieval) operations through a series of well-defined stages.

The process begins with the input module, where the user provides the secret data that needs to be protected. This data can be in the form of text or an image. Along with this, the user also selects a cover image, which acts as a medium to hide the secret information. The cover image plays a crucial role, as its quality and size directly affect the efficiency of the data hiding process.

The next stage is the steganography module, where the secret data is embedded into the cover image using the Least Significant Bit (LSB) technique. In this method, the least significant bits of the image pixels are modified to store the secret data. Since these changes are minimal, the human eye cannot detect any difference between the original image and the

modified image. The output of this stage is called the stego image, which contains the hidden data.

After generating the stego image, the system moves to the visual cryptography module, which provides the second layer of security. In this stage, the stego image is divided into multiple shares (such as two or more shares). Each share appears as random noise and does not reveal any meaningful information individually. These shares are then stored or transmitted separately, ensuring that unauthorized users cannot access the hidden data even if they obtain one of the shares.

For the decoding process, the system first requires the correct set of shares. In the share combination module, the shares are combined to reconstruct the original stego image. This step is crucial because reconstruction is only possible when all required shares are available, ensuring secure access control.

Once the stego image is reconstructed, it is passed to the data extraction module, where the hidden data is retrieved using the reverse LSB technique. The system carefully extracts the embedded bits from the image pixels and reconstructs the original secret data. This ensures accurate recovery of the information without loss.

Additionally, the system includes a performance evaluation module, where metrics such as Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index Measure (SSIM) are calculated. These metrics help in analyzing the quality of the stego image and ensure that the embedding process does not significantly affect the visual appearance of the image.

Finally, the system provides a user interface module developed using Streamlit, which allows users to easily perform encoding, share generation, decoding, and result visualization.

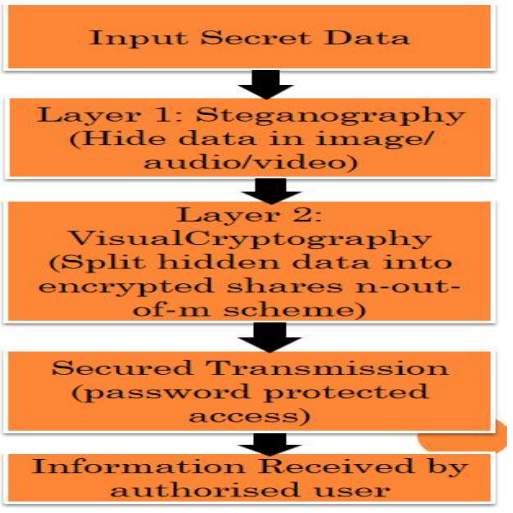

Fig 3.5: Block diagram of the system

4. SYSTEM DESIGN

4.1 Introduction

The system design of this project focuses on developing a secure and efficient framework for multilevel data concealing using steganography and visual cryptography techniques. The main aim of the system is to protect sensitive information by hiding it in such a way that unauthorized users cannot detect or access it. Unlike traditional methods that rely only on encryption, this system enhances security by combining multiple techniques to provide a stronger and more reliable protection mechanism.

In the first level of security, steganography is used to embed secret data into a digital image. This process ensures that the presence of hidden information is not visible to the human eye, making it difficult for attackers to identify that any data is concealed within the image. The system carefully modifies pixel values (such as RGB components) to store the secret message without significantly affecting the image quality.

To further enhance security, the system applies visual cryptography as a second layer of protection. In this process, the encoded image or hidden data is divided into multiple shares. Each share appears meaningless on its own, and the original information can only be reconstructed when the required number of shares are combined. This ensures that even if one share is intercepted, the data remains secure and cannot be accessed.

The system design also includes a user-friendly interface that allows users to perform encoding and decoding operations easily. Users can upload images, enter secret data, generate shares, and reconstruct the original information without requiring technical expertise. Additionally, the system is structured into modules such as user authentication, data hiding, share generation, and data reconstruction, ensuring a well-organized and modular architecture.

Moreover, the system incorporates performance evaluation metrics such as Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM) to measure the quality of the processed images. These metrics help in ensuring that the hidden data does not significantly degrade the visual quality of the image. Overall, the system design provides a secure, reliable, and scalable solution for data protection, making it suitable for applications like secure communication, confidential data storage, and digital forensics.

The system also focuses on providing a user-friendly interface using Streamlit, enabling users to easily interact with the application.

4.2 UML Diagrams

UML, which stands for Unified Modelling Language, is a standard way to visually represent the architecture, design, and implementation of software systems. Understanding a system only through code can be difficult, so UML diagrams are used to provide a clear and simplified visualization of system components and their interactions. These diagrams help both technical and non-technical users to easily understand how the system works. UML is a pictorial language used for designing and documenting software systems and is different from programming languages such as C++, Java, or Python.

In this project, UML diagrams are used to represent the structure and workflow of the multilevel data concealing system. The system begins with the user providing secret data and selecting a cover image. The steganography module embeds the secret data into the image using the LSB technique, producing a stego image. This image is then processed by the visual cryptography module, where it is divided into multiple shares for enhanced security. These shares are stored or transmitted separately to ensure that no individual share reveals any information.

During the decoding phase, the required shares are combined to reconstruct the original stego image. The system then extracts the hidden data using the reverse steganography process. The backend processing is handled using Python, while the user interacts with the system through a Streamlit-based interface. The system ensures secure data handling, accurate reconstruction, and efficient processing at each stage.

UML diagrams such as use case diagrams, sequence diagrams, activity diagrams, and class diagrams can be used to model the structure and behavior of the system. These diagrams help in visualizing the flow of data, interaction between modules, and overall system functionality. Thus, UML plays an important role in designing and understanding the multilevel data concealing system effectively.

UML diagrams improve clarity by simplifying complex system processes into easy-to-understand visuals. They help in identifying design issues early, reducing development errors. UML also enhances communication among developers by providing a common representation of the system. It supports proper documentation, which is useful for future modifications and maintenance. Overall, it contributes to building a more structured and efficient system.

4.2.1 Class Diagram

Because a lot of software is based on object-oriented programming, where developers define types of functions that can be used, class diagrams are the most commonly used type of UML diagram. Class diagrams show the static structure of a system, including classes, their attributes and behaviours, and the relationships between each class. A class is represented by a rectangle that contains three compartments stacked vertically: Class name, attributes and functions of that class with access specifiers.The class diagram also contains association, generalization and aggregation.Where generalization is to show inheritance of a class and aggregation is to show that objects are assembled or configured together to create a more complex object.

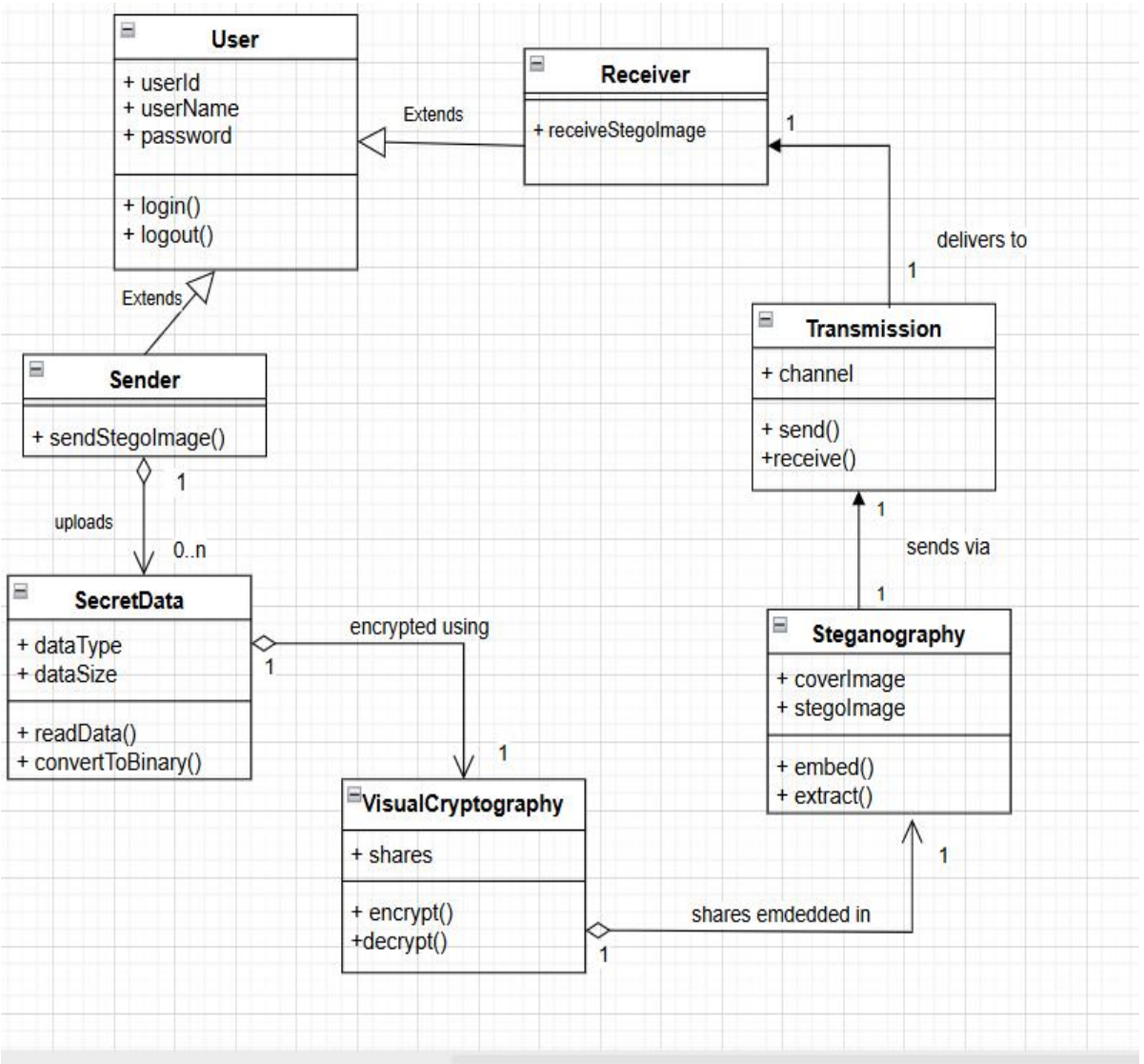

Fig 4.2.1: Class diagram

4.2.2 Activity Diagram

The Unified Modelling Language includes several subsets of diagrams, including structure diagrams, interaction diagrams, and behaviour diagrams. Activity diagrams, along with usecase and state machine diagrams, are considered behaviour diagrams because they describe what must happen in the system being modelled. Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system. The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent.Activity diagrams deal with all type of flow control by using different elements such as fork, join, etc.

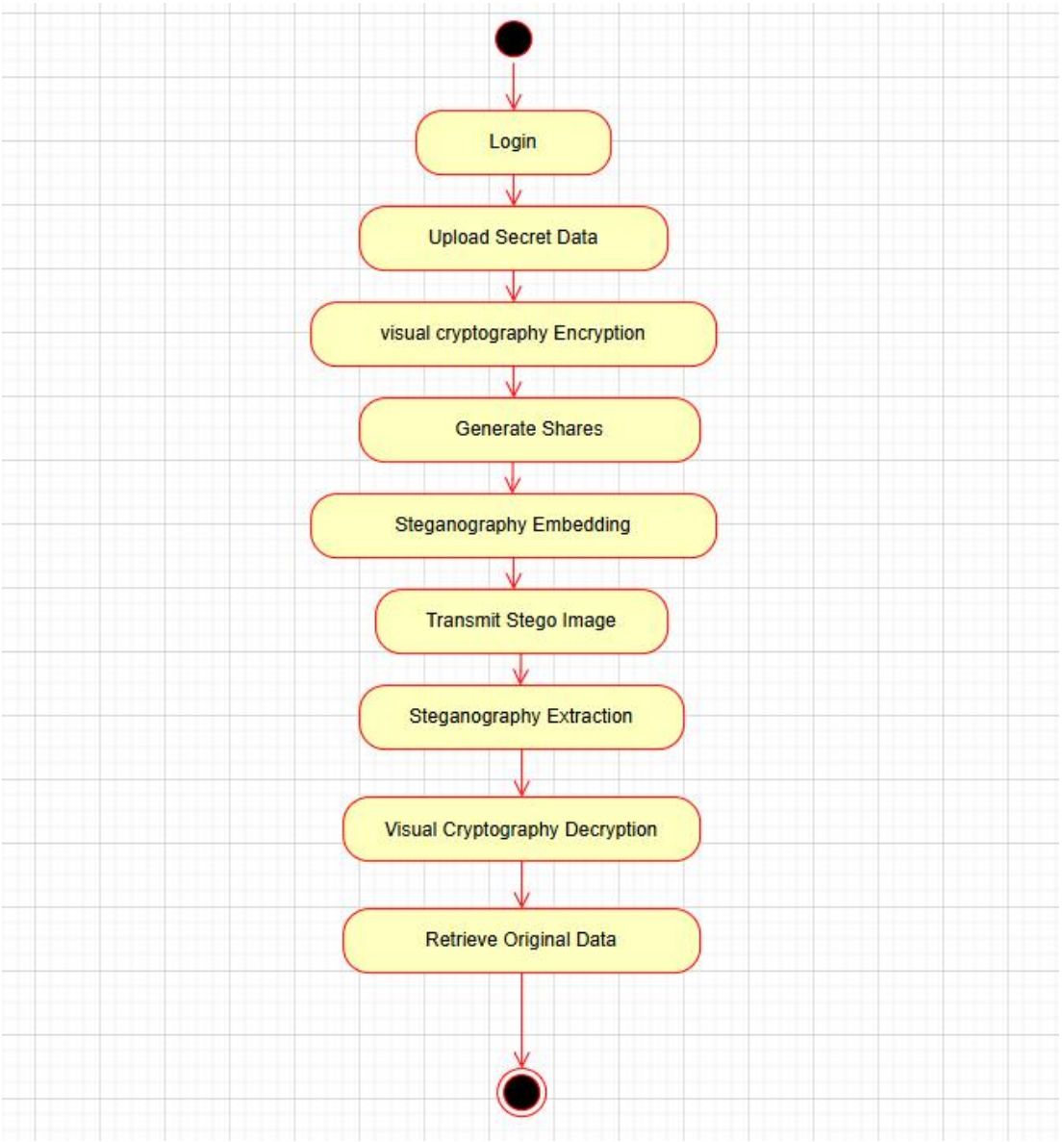

Fig 4.2.2: Activity diagram

4.2.3 Usecase Diagram

A UML use case diagram is the primary form of system/software requirements for a new software program under developed. In the Unified Modelling Language (UML), a use case diagram can summarize the details of your system's users (also known as actors) and their interactions with the system.To build one, you'll use a set of specialized symbols and connectors. It is useful in the following situations: Scenarios in which your system or application interacts with people, organizations, or external systems, Goals that your system or application helps those entities (known as actors) achieve, The scope of your system. Use cases specify the expected behaviour (what) ,and not the exact method of making it happen (how).

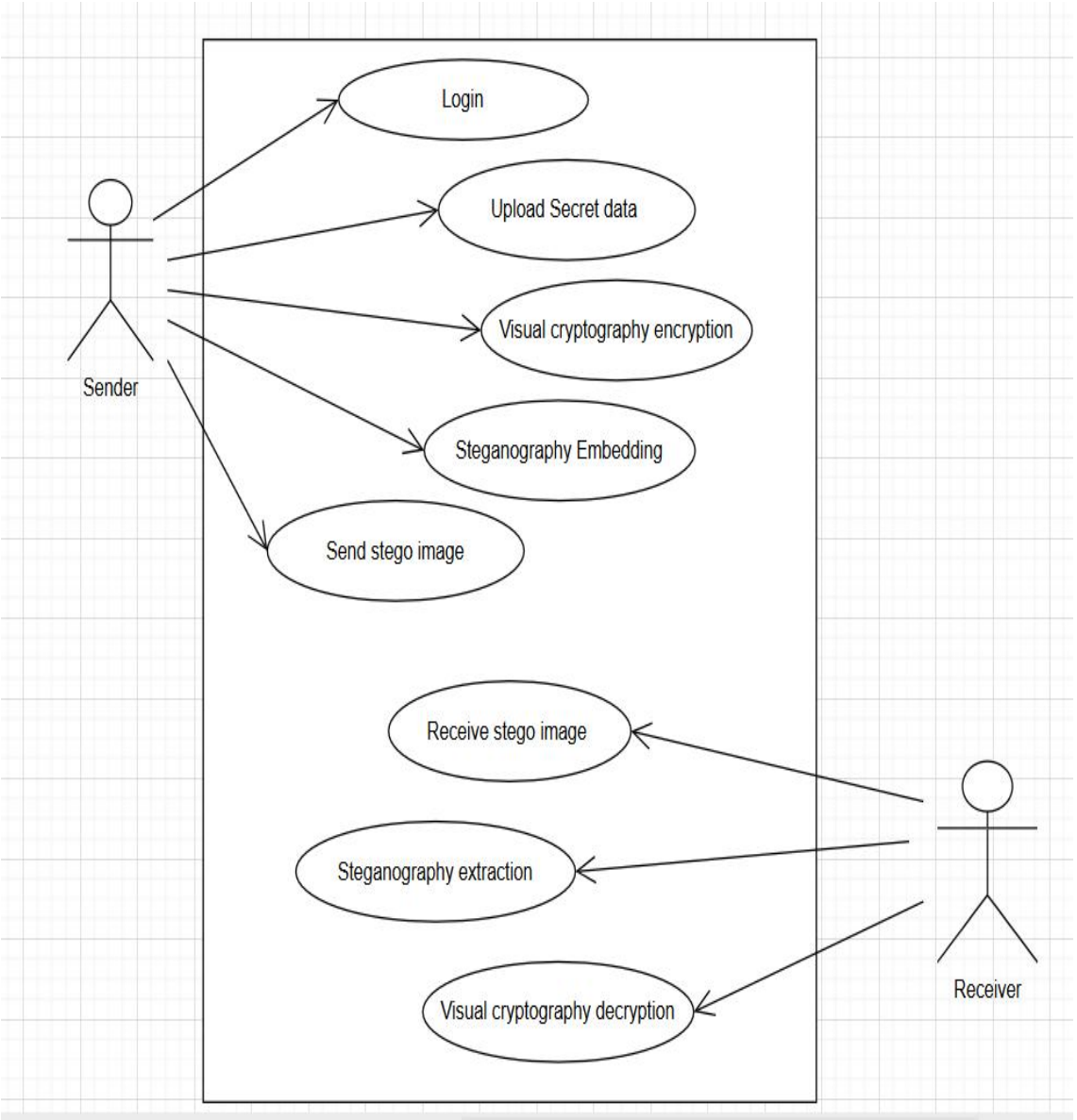

Fig 4.2.3: Usecase diagram

4.2.4 Sequence Diagram

UML Sequence Diagrams are interaction diagrams that detail how operations are carried out.They capture the interaction between objects in the context of a collaboration, A sequence diagram is a type of interaction diagram because it describe show and in what order a group of objects works together. These diagrams are used by software developers and business professionals to understand requirements for a new system or to document an existing process. Sequence diagrams are sometimes known as event diagrams or event scenarios.

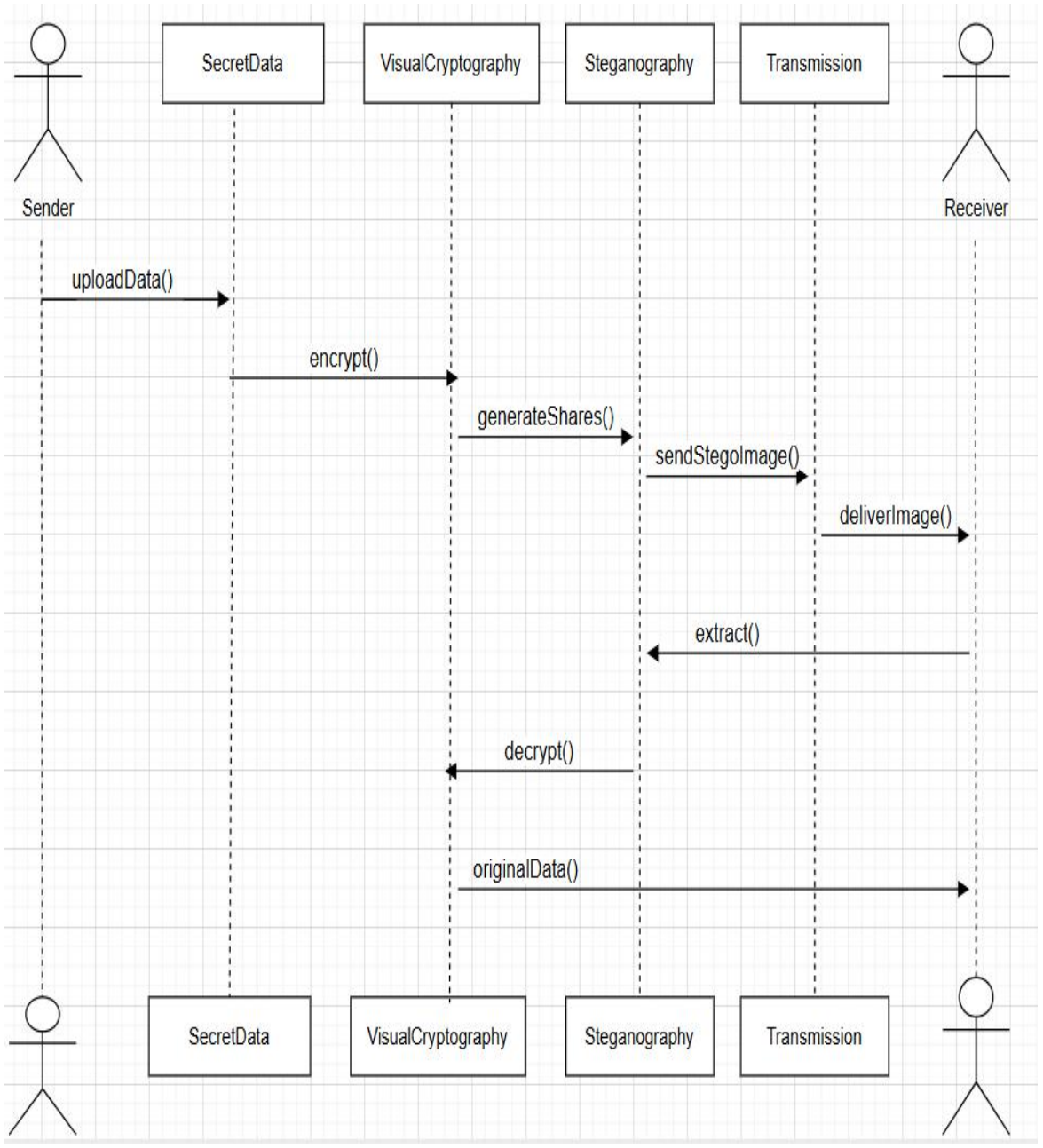

Fig 4.2.4: Sequence diagram

4.3 Modules

The proposed Multilevel Data Concealing System using Steganography and Visual Cryptography is divided into multiple modules to ensure proper organization, easy implementation, and efficient performance. Each module is responsible for a specific task and works together to provide secure data hiding and retrieval.

4.3.1 User Interface Module:

The User Interface Module provides an interactive platform for users to interact with the system. This module is developed using Streamlit, allowing users to easily upload images, enter secret data, and perform encoding and decoding operations.

Through this interface, users can choose between hiding or extracting data, upload cover images, and view the generated outputs such as stego images and shares. The module ensures simplicity and ease of use so that even non-technical users can operate the system effectively.

4.3.2 Steganography Module:

The Steganography Module is responsible for hiding secret data inside a cover image. This module uses the Least Significant Bit (LSB) technique to embed the data into the pixel values of the image without causing noticeable changes.

The input image is converted into pixel format (RGB), and the secret message is encoded into the least significant bits of these pixels. The output of this module is a stego image, which looks visually similar to the original image but contains hidden information.

4.3.3 Visual Cryptography Module:

The Visual Cryptography Module enhances security by splitting the stego image into multiple shares. Each share appears as random noise and does not reveal any meaningful information individually.

The module generates shares based on visual cryptography principles, ensuring that only when the required number of shares are combined, the original stego image can be reconstructed. This provides an additional layer of protection against unauthorized access.

4.3.4 Share Management Module:

The Share Management Module handles the storage and transmission of generated shares. It ensures that shares are stored securely and can be retrieved when needed for decoding.

This module may store shares locally or allow users to download them. Proper handling of shares is important because losing any required share will make it impossible to reconstruct the original data.

4.3.5 Data Reconstruction and Extraction Module:

This module is responsible for decoding the hidden data. First, it combines the required shares to reconstruct the original stego image using visual cryptography techniques.

After reconstruction, the module applies reverse steganography to extract the hidden message from the stego image. The extracted data is then displayed to the user accurately. This module ensures correct data recovery and completes the secure communication process.

4.3.6 Performance Evaluation Module:

The Performance Evaluation Module is used to measure the quality of the stego image. It calculates metrics such as PSNR (Peak Signal-to-Noise Ratio) and SSIM (Structural Similarity Index).

These metrics help in evaluating how much the image quality is affected after embedding the data. High PSNR and SSIM values indicate that the stego image is very similar to the original image, ensuring better invisibility of hidden data.

5. IMPLEMENTATION

5.1 Introduction

The implementation of the Multilevel Data Concealing System using Steganography and Visual Cryptography focuses on developing a secure and efficient application that ensures safe transmission and storage of sensitive information. The system is implemented using Python, integrating image processing techniques and cryptographic methods to provide a two-level security mechanism. The overall implementation is designed in a modular manner, where each component performs a specific function, ensuring better maintainability and scalability.

The frontend of the system is developed using Streamlit, which provides a simple and interactive web-based interface for users. Through this interface, users can upload cover images, input secret data, and perform encoding and decoding operations with ease. The backend logic is implemented using Python libraries such as OpenCV, NumPy, and PIL, which are used for image processing, pixel manipulation, and efficient data handling.

During the encoding process, the system first converts the input image into pixel format and applies the Least Significant Bit (LSB) steganography technique to embed the secret data into the image. The modified image, known as the stego image, is then passed to the visual cryptography module, where it is divided into multiple shares. Each share is generated in such a way that it does not reveal any meaningful information independently, thereby enhancing data security.

In the decoding phase, the system requires the appropriate shares to reconstruct the original stego image. Once the shares are combined, the reverse steganography process is applied to extract the hidden data from the image. The system ensures that the extracted data is accurate and matches the original input provided by the user.

Additionally, the implementation includes performance evaluation techniques such as PSNR and SSIM to measure the quality of the stego image and ensure minimal distortion. Proper error handling and validation mechanisms are also incorporated to manage invalid inputs and ensure smooth system operation.

Overall, the implementation provides a reliable, user-friendly, and secure solution for multilevel data hiding and retrieval.

5.2 Method of Implementation

The Agile methodology is a project management approach that divides the development process into smaller phases, allowing continuous improvement and flexibility. It emphasizes collaboration, iterative development, and regular feedback at every stage. Instead of building the entire system at once, the project is developed step by step through planning, execution, and evaluation cycles. In this project, Agile methodology is followed to ensure efficient development, better quality, and adaptability to changes during the implementation of the multilevel data concealing system.

The project was implemented using Agile principles, where the system is divided into smaller modules such as the user interface, steganography, visual cryptography, share management, and data extraction. Each module was developed and tested individually before integrating it into the complete system. This approach helped in identifying errors early and improving the performance of each module step by step. Agile is particularly suitable for this project because it allows continuous testing and refinement of image processing and data hiding techniques.

In this project, the development was carried out in multiple stages. Initially, the basic system setup was created using Python and Streamlit to build the user interface. In the next stage, the steganography module was implemented using the LSB technique to hide secret data within images. After that, the visual cryptography module was developed to split the stego image into multiple secure shares. Finally, the decoding process was implemented to combine shares and extract the hidden data accurately. Each stage was tested and improved before moving to the next stage.

Using Agile methodology helped in improving system accuracy, reducing errors, and simplifying debugging. It also made it easier to integrate different components such as image processing, data hiding, and share generation. Continuous testing ensured that the quality of the stego image was maintained while preserving the hidden data. This approach resulted in a more reliable and efficient system.

Agile's four main values are:

- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

Fig 5.2: Agile method

5.3 Algorithms and Flowcharts

The proposed system utilizes image processing and cryptographic techniques to securely hide and protect sensitive data using a multilevel approach. The system operates by combining steganography and visual cryptography to ensure that the secret data remains both invisible and highly secure. The primary objective of the algorithm is to embed confidential information within an image and further protect it by dividing it into multiple shares, making unauthorized access extremely difficult.

The data hiding process is carried out using the Least Significant Bit (LSB) steganography technique, which is widely used for its simplicity and effectiveness. In this method, the secret data is embedded into the least significant bits of the image pixels without causing noticeable changes to the image. The system processes the image using Python libraries such as OpenCV and NumPy, which handle pixel manipulation and image processing efficiently. The output of this stage is a stego image that appears visually identical to the original image but contains hidden information.

To enhance security, the system applies visual cryptography to the stego image. In this process, the stego image is divided into multiple shares, where each share appears as random noise and does not reveal any meaningful data on its own. Only when the required number of shares are combined can the original stego image be reconstructed. This ensures that even if one share is intercepted, the hidden data remains secure.

During the decoding phase, the system reconstructs the stego image by combining the required shares. After reconstruction, the reverse steganography process is applied to extract the hidden data from the image. The system ensures accurate recovery of the original message without loss of information. The entire workflow is managed through a user-friendly interface developed using Streamlit, allowing users to easily perform encoding and decoding operations.

Additionally, the system evaluates the quality of the stego image using performance metrics such as PSNR and SSIM to ensure minimal distortion. Proper validation and error handling mechanisms are included to handle incorrect inputs or missing shares. Overall, the algorithm provides a secure, reliable, and efficient method for multilevel data protection.

Algorithm:

Encoding Process:

1. Start the system: Initialize the application and load required libraries such as OpenCV, NumPy, and PIL. Launch the Streamlit interface.
2. Upload cover image and enter secret data: The user selects an input image and provides the secret message to be hidden inside the image.
3. Validate inputs: Check whether the image is uploaded and the secret data is not empty. Also ensure the image has sufficient capacity to store the data.
4. Convert secret data into binary format: Transform each character of the secret message into its ASCII value and then into binary bits for embedding.
5. Preprocess the image: Convert the image into pixel format (RGB channels) and extract pixel values for manipulation.
6. Apply LSB steganography: Replace the least significant bits of pixel values with the binary data of the secret message. This ensures minimal visual distortion.
7. Generate stego image: Reconstruct the image from modified pixel values and produce the stego image containing hidden data.
8. Apply visual cryptography: Convert the stego image into shares using visual cryptography techniques. Each share is generated such that it appears random and meaningless individually.
9. Generate multiple shares: Create two or more shares depending on the scheme (e.g., 2-out-of-2 scheme).
10. Store or download shares: Allow users to securely save or download the generated shares for future reconstruction.

Decoding Process:

11. Upload required shares: The user uploads the necessary shares into the system for reconstruction.
12. Validate shares: Ensure the correct number and type of shares are provided. If any share is missing or incorrect, stop the process.
13. Combine shares: Overlay or merge the shares using visual cryptography rules to reconstruct the original stego image.
14. Extract pixel data: Read the pixel values from the reconstructed stego image.
15. Apply reverse LSB technique: Extract the least significant bits from the pixel values to retrieve the binary data.
16. Convert binary to text: Group binary bits into bytes and convert them back into characters.
17. Detect termination indicator: Stop extraction when the delimiter is encountered to avoid unwanted data.
18. Display extracted data: Show the recovered secret message to the user through the interface.
19. Stop the system: End the process and release resources.

5.4 Control Flow of the Implementation

The control flow of the system describes how the execution moves step by step from user input to final output during both encoding and decoding processes. The system starts by initializing the application and loading all required libraries and modules. Once the system is ready, the user interacts with the interface to choose the desired operation, either encoding (hiding data) or decoding (extracting data).

In the encoding phase, the control first moves to input validation, where the system checks whether the user has uploaded a valid image and entered the secret data. If the inputs are valid, the system proceeds to convert the secret data into binary format. Then, the steganography module is executed, where the binary data is embedded into the image pixels using the LSB technique. After embedding, the system generates the stego image.

Next, the control passes to the visual cryptography module, where the stego image is divided into multiple shares. These shares are then provided to the user for storage or transmission. Once the shares are generated successfully, the encoding process is completed.

In the decoding phase, the control begins again with user input, where the user uploads the required shares. The system validates whether all necessary shares are provided. If

validation is successful, the control moves to the share combination module, where the shares are merged to reconstruct the original stego image.

After reconstruction, the control is transferred to the data extraction module. Here, the system applies the reverse LSB process to retrieve the hidden binary data from the image. The binary data is then converted back into readable text. Once the extraction is complete, the secret message is displayed to the user.

Throughout the process, error handling and validation checks are performed at each stage to ensure smooth execution. If any error occurs, such as invalid input or missing shares, the system stops the current process and prompts the user to correct the input. Finally, the system completes execution and waits for the next user action.

5.4.1 Importing Dependencies

Importing dependencies is the first step in the implementation of the proposed system. In this step, all the required libraries and modules needed for the multilevel data concealing system are imported into the program. These libraries provide essential functionalities such as image processing, data manipulation, user interface handling, and secure data storage.

The system is implemented using Python, where libraries such as OpenCV (cv2) and PIL (Python Imaging Library) are used for image processing tasks like reading, modifying, and saving images. The NumPy library is used for efficient handling of image pixel data and performing numerical operations. The Streamlit framework is used to build an interactive web-based interface that allows users to upload images, enter secret data, and perform encoding and decoding operations easily.

Additional modules are used to implement the core functionalities of the system, such as hiding text within images, extracting hidden data, generating visual cryptographic shares, and combining shares during the decoding process. The system may also include utility functions for calculating performance metrics like PSNR and SSIM to evaluate image quality. These dependencies work together to ensure smooth execution, efficient processing, and secure handling of data throughout the system.

Furthermore, proper dependency management helps in maintaining code modularity and reusability, making the system easier to update and extend in the future. The use of well-established libraries also improves the reliability and performance of the application.Each imported module plays a specific role, contributing to the overall functionality of the system.

Thus, importing dependencies forms the foundation of the implementation and ensures that all required tools are available for building a robust and efficient data concealing system.

```
import streamlit as st
import numpy as np
import cv2
from PIL import Image
import io
import os

from utils_fixed import (
    hide_text_in_image,
    reveal_text_from_image,
    hide_image_in_image,
    reveal_image_from_image,
    create_shares,
    combine_shares,
    calculate_psnr,
    calculate_ssim
)

from database import init_db, register_user, validate_login
```

Fig 5.4.1: Importing dependencies

5.5 Sample Code

The sample code demonstrates the implementation of a multilevel data concealing system using image processing and cryptographic techniques. It integrates data hiding using the Least Significant Bit (LSB) steganography method and enhances security through visual cryptography by splitting the encoded image into multiple shares. The system provides both encoding and decoding functionalities, allowing users to securely hide and retrieve sensitive information. Additionally, it includes mechanisms for share generation, reconstruction, and

performance evaluation, ensuring secure data transmission and reliable recovery of the hidden message.

1. Main.py

```
import streamlit as st
from PIL import Image
import tempfile
import os

from utils_fixed import (
    hide_text_in_image, reveal_text_from_image,
    create_shares, combine_shares
)

st.title(" Data Conceal System")

tab1, tab2, tab3 = st.tabs(["Encode", "Decode", "Visual Crypto"])

# ------------------ ENCODE ------------------
with tab1:
    st.header("Hide Text in Image")
    cover = st.file_uploader("Upload Cover Image", type=['png','jpg'])
    text = st.text_area("Enter Secret Text")

    if st.button("Encode") and cover and text:
        with tempfile.NamedTemporaryFile(delete=False) as inp, \
             tempfile.NamedTemporaryFile(delete=False) as out:

            inp.write(cover.read())
            success, msg = hide_text_in_image(inp.name, text, out.name)

            if success:
                st.image(out.name)
                st.download_button("Download", open(out.name, "rb"), "stego.png")
            else:
                st.error(msg)

# ------------------ DECODE ------------------
with tab2:
    st.header("Reveal Text")
    stego = st.file_uploader("Upload Stego Image", type=['png','jpg'])

    if st.button("Decode") and stego:
        with tempfile.NamedTemporaryFile(delete=False) as tmp:
            tmp.write(stego.read())
            success, msg = reveal_text_from_image(tmp.name)
```

```
            if success:
                st.text_area("Hidden Text", msg)
            else:
                st.error(msg)

# ----------------VISUAL CRYPTOGRAPHY-----------
with tab3:
    st.header("Visual Cryptography")

    img = st.file_uploader("Upload Image", type=['png','jpg'])

    if st.button("Split into Shares") and img:
        with tempfile.NamedTemporaryFile(delete=False) as tmp:
            tmp.write(img.read())
            success, shares = create_shares(tmp.name, 2)

            if success:
                for i, s in enumerate(shares):
                    st.image(s, caption=f"Share {i+1}")
            else:
                st.error(shares)

    files = st.file_uploader("Upload Shares", accept_multiple_files=True)

    if st.button("Reconstruct") and files:
        paths = []
        for f in files:
            t = tempfile.NamedTemporaryFile(delete=False)
            t.write(f.read())
            paths.append(t.name)

        success, img = combine_shares(paths)

        if success:
            st.image(img)
        else:
            st.error(img)
```

The given code implements a basic Data Concealing System using the Streamlit framework. It provides three main functionalities: hiding text in an image, extracting hidden text, and performing visual cryptography operations.

At the beginning, necessary libraries such as Streamlit, PIL, and tempfile are imported. Custom functions are also imported from a utility file to handle core operations like hiding text, revealing text, splitting images into shares, and combining shares. These functions perform the main steganography and visual cryptography tasks.

The application interface is created using Streamlit and is divided into three tabs: Encode, Decode, and Visual Cryptography. This helps users easily switch between different features.

In the Encode section, the user uploads a cover image and enters secret text. When the encode button is clicked, the system hides the text inside the image using the steganography function. The output image (stego image) is displayed and can be downloaded.

In the Decode section, the user uploads a stego image. When the decode button is clicked, the system extracts the hidden text from the image and displays it on the screen.

In the Visual Cryptography section, the user can upload an image and split it into shares. These shares are displayed individually and do not reveal any information on their own. The user can also upload multiple shares and reconstruct the original image by combining them.

2. Login.py

```
import sqlite3, hashlib

DB = 'users.db'

def init_db():
    with sqlite3.connect(DB) as conn:
        conn.execute('CREATE TABLE IF NOT EXISTS users (username TEXT UNIQUE,
            password TEXT)')

def hash_pass(p):
    return hashlib.pbkdf2_hmac('sha256', p.encode(), b'salt', 100000).hex()

def register_user(u, p):
    init_db()
    try:
        with sqlite3.connect(DB) as conn:
            conn.execute('INSERT INTO users VALUES (?, ?)', (u, hash_pass(p)))
        return True, "Registered"
    except:
        return False, "User exists"

def validate_login(u, p):
    init_db()
    with sqlite3.connect(DB) as conn:
        cur = conn.execute('SELECT * FROM users WHERE username=? AND password=?', (u,
            hash_pass(p)))
        return (True, "Login success") if cur.fetchone() else (False, "Invalid login")
```

The code implements a basic user authentication system using Python and SQLite. It allows users to register and log in securely by storing hashed passwords instead of plain text passwords.

At the beginning, the required libraries sqlite3 and hashlib are imported. The database file is defined as *users.db*, which is used to store user credentials.

The init_db() function is responsible for creating the database table named *users* if it does not already exist. This table stores the username and password for each user.

The hash_pass() function is used to securely convert the user's password into a hashed format using the algorithm with SHA-256. This improves security by preventing direct storage of passwords.

The register_user() function is used to create a new user account. It first ensures the database is initialized, then inserts the username and hashed password into the database. If the username already exists, it returns an error message.

The validate_login() function checks whether the entered username and password match any record in the database. It hashes the entered password and compares it with the stored value. If a match is found, login is successful; otherwise, it returns an invalid login message.

3. Utils.py

```
import numpy as np
from PIL import Image

# -------- TEXT STEGANOGRAPHY --------

def hide_text(img_path, text, out):
    img = Image.open(img_path).convert('RGB')
    data = ''.join(format(ord(c), '08b') for c in text) + '1111111111111110'
    pixels = np.array(img).flatten()

    for i, bit in enumerate(data):
        pixels[i] = (pixels[i] & ~1) | int(bit)

    Image.fromarray(pixels.reshape(img.size[1], img.size[0], 3)).save(out)
    return True

def reveal_text(img_path):
    img = Image.open(img_path).convert('RGB')
    pixels = np.array(img).flatten()
    bits = ''.join(str(p & 1) for p in pixels)

    chars = [bits[i:i+8] for i in range(0, len(bits), 8)]
```

```
    text = ''
    for c in chars:
        if c == '11111110': break
        text += chr(int(c, 2))
    return text

# -------- IMAGE STEGANOGRAPHY --------

def hide_image(cover, secret, out):
    c = np.array(Image.open(cover).convert('RGB'))
    s = np.array(Image.open(secret).resize((c.shape[1], c.shape[0])))

    stego = (c & 0xF0) | (s >> 4)
    Image.fromarray(stego).save(out)
    return True

def reveal_image(stego, out):
    s = np.array(Image.open(stego))
    Image.fromarray((s & 0x0F) << 4).save(out)
    return True

# -------- VISUAL CRYPTOGRAPHY --------

def create_shares(path):
    img = np.array(Image.open(path).convert('L')) > 128
    share1 = np.random.randint(0, 2, img.shape)
    share2 = share1 ^ img

    return [Image.fromarray(share1*255), Image.fromarray(share2*255)]

def combine_shares(s1, s2):
    a = np.array(Image.open(s1)) > 128
    b = np.array(Image.open(s2)) > 128

    return Image.fromarray((a ^ b)*255)
```

The code is divided into three main parts: text steganography, image steganography, and visual cryptography.

In the text steganography section, the hide_text() function is used to hide secret text inside an image. It converts the text into binary format and appends a special delimiter to mark the end of the message. Then, it modifies the least significant bits (LSB) of the image pixels to store the binary data. The reveal_text() function performs the reverse operation by extracting the LSB bits from the image, converting them back into binary, and then reconstructing the original text until the delimiter is found.

In the image steganography section, the hide_image() function hides one image inside another. It does this by clearing the lower bits of the cover image and inserting the higher bits of the secret image into them. The reveal_image() function extracts the hidden image by retrieving and shifting the stored bits back to reconstruct the secret image.

In the visual cryptography section, the create_shares() function splits an image into two shares. It converts the image into a binary format and generates one random share. The second share is created using an XOR operation between the first share and the original image. Individually, these shares do not reveal any information. The combine_shares() function combines the two shares using XOR operation to reconstruct the original image.

6. TESTING

6.1 Introduction to Testing

Software testing is a process, to evaluate the functionality of a software application with an intent to find whether the developed software met the specified requirements or not and to identify the defects to ensure that the product is defect free to produce the quality product.

Testing is an important phase in software development that ensures the system functions correctly and meets the required specifications. It involves evaluating the application to identify errors, verify system performance, and ensure that all modules operate as expected. Proper testing helps improve system reliability, accuracy, and overall quality.

Testing is an essential phase in the development of the Multilevel Data Concealing System using Steganography and Visual Cryptography. It ensures that the system functions correctly, securely, and efficiently under different conditions. The main objective of testing is to identify errors, verify system performance, and ensure that all modules work together as expected.

In this project, testing is performed on various components such as the steganography module, visual cryptography module, and data extraction module. Each module is tested individually to check its functionality, such as correct embedding of secret data, accurate generation of shares, and proper reconstruction of the original image. After unit testing, integration testing is carried out to ensure smooth interaction between all modules.

The encoding process is tested by verifying whether the secret data is successfully hidden inside the image without noticeable changes in image quality. Similarly, the decoding process is tested to ensure that the hidden data can be accurately retrieved from the stego image. Visual cryptography is tested by checking whether individual shares reveal no information and whether the original image can be correctly reconstructed when the shares are combined.Additionally, performance testing is conducted using metrics such as PSNR (Peak Signal-to-Noise Ratio) and SSIM (Structural Similarity Index) to evaluate the quality of the processed images. High values of these metrics indicate that the system preserves image quality while hiding data.

Overall, testing plays a crucial role in ensuring the reliability, security, and efficiency of the system, making it suitable for real-world applications involving secure data transmission and storage.

6.2 Types of Tests Considered

- **Unit Testing:**

Unit testing focuses on testing individual modules of the system separately to ensure each component works correctly. In this project, modules such as text steganography, image steganography, visual cryptography (share generation), and data extraction were tested independently.

For example, the text hiding module was tested to verify whether secret text is correctly embedded into the image without errors. Similarly, the reveal module was tested to ensure accurate extraction of hidden data. The visual cryptography module was also tested to check whether shares are generated properly. Unit testing helps in identifying errors early and simplifies debugging.

- **Integration Testing:**

Integration testing ensures that different modules of the system work together properly. In this project, it verifies the interaction between the steganography module and visual cryptography module.

For instance, after hiding data in an image (stego image), the system should correctly pass it to the visual cryptography module to generate shares. During decoding, the shares should combine correctly and allow successful data extraction. Integration testing ensures smooth data flow and eliminates errors between modules.

- **Functional Testing:**

Functional testing checks whether the system performs all its intended operations according to requirements. It ensures that all features are working correctly under normal conditions.

In this project, functional testing includes verifying whether the system can hide text or images, generate shares, reconstruct the image, and extract the hidden data accurately. It also ensures that user inputs, file uploads, and outputs in the Streamlit interface function properly.

- **System Testing:**

System testing evaluates the complete system as a whole to ensure all components work together correctly. It checks the overall behavior of the application in a real-time environment.
In this project, the entire workflow—from uploading an image, hiding data, generating shares,

reconstructing the image, and extracting data—is tested. This ensures the system works correctly in real-world usage scenarios.

- **Performance Testing:**

Performance testing evaluates the efficiency and speed of the system. It ensures that the system processes data quickly and maintains good performance.

In this project, performance testing is done by measuring how efficiently the system hides and retrieves data without affecting image quality. Metrics like PSNR and SSIM are used to evaluate the quality of stego and reconstructed images. This testing ensures that the system is reliable and suitable for practical use.

6.3 Various Test Cases Considered

Test case writing is a major activity and is considered one of the most important parts of software testing. It is used by the testing team, development team, and management to ensure that the system functions correctly and meets the required specifications. If proper documentation is not available, test cases can act as a baseline document to understand the system behavior and validate its functionality.

In the proposed, several test cases were designed and executed to ensure the accuracy, security, and reliability of the system. These test cases cover different scenarios related to data hiding, data extraction, and share generation.

Test cases were created to verify the text steganography process, where different types of input such as short text, long text, and special characters were hidden inside images. The system was tested to ensure that the hidden data is correctly embedded and can be accurately retrieved without loss of information.

Similarly, test cases were designed for image steganography, where one image is hidden inside another. The system was tested with images of different sizes and formats to ensure proper embedding and extraction. It was also verified that the quality of the cover image is not significantly affected.

For the visual cryptography module, test cases were conducted to check whether the image is correctly divided into multiple shares. Each share was tested individually to confirm that it does not reveal any meaningful information. Additional test cases ensured that combining the correct shares reconstructs the original image accurately, while incorrect or missing shares fail to produce valid output.

Table 6.3: Test cases for proposed system

TEST CASE ID	TESTCASE SCENARIO	INPUTS	EXPECTED OUTPUT	ACTUAL OUTPUT	STATUS
1	Login with by giving user name and password	Enter email and password	If given the credentials are correct,it should redirect to menupage or else you have to repeat the process.	The menu page is displayed successfully if the credentials are correct.	Pass
2	Navigate to Encode Page	Click on “Encode” option	Encode page should open with options to hide text/image	Encode page opens successfully	Pass
3	Hide Text in Image	Enter secret text and select cover image	Text should be embedded in the image and saved	Text hidden successfully in the image	Pass
4	Hide Image in Image	Select secret image and cover image	Secret image should be hidden inside cover image	Image hidden successfully	Pass
5	Visual Cryptography	Upload image to generate shares	Image is split into shares that look like random noise	Shares generated successfully	Pass
6	Decode Hidden Text	Upload stego image	Hidden text should be extracted and displayed	Text extracted successfully	Pass
7	Decode Hidden Image	Upload stego image	Hidden image should be revealed	Image revealed successfully	Pass
8	Quality Metrics	Upload original and stego/revealed image	Display PSNR and SSIM values to show quality	PSNR and SSIM values displayed correctly	Pass

The testing of the system began with verifying the login functionality, where users enter their username and password. The system was expected to redirect users to the main menu page if the credentials were correct. The results confirmed that valid credentials successfully allowed access, ensuring proper authentication. Next, the navigation feature was tested by selecting the “Encode” option, which correctly opened the encode page with options to hide text or images, confirming smooth user interface flow.

Further testing focused on the core encoding functionalities. In the text steganography test case, users entered secret text and selected a cover image, and the system successfully embedded the text into the image. Similarly, in the image steganography test case, a secret image was hidden inside a cover image, and the output confirmed successful data concealment. The visual cryptography module was also tested by uploading an image and generating shares. The system correctly split the image into shares that appeared as random noise, ensuring that no individual share revealed any meaningful information.

The decoding functionalities were then tested to verify data retrieval. The system successfully extracted hidden text from stego images and accurately revealed hidden images, confirming the correctness of the decoding process. Finally, the quality metrics feature was tested by uploading original and processed images, where the system correctly displayed PSNR and SSIM values to evaluate image quality. All test cases produced expected outputs, and the system performed reliably across all scenarios, indicating that it is accurate, efficient, and suitable for secure data concealing applications.

7. RESULT AND OUTPUT SCREENS

The results of the proposed system show that the system works effectively for secure data hiding and sharing. The system allows users to hide text or images inside a cover image using the LSB technique without noticeable changes in image quality.

The generated stego image is further divided into multiple shares using visual cryptography, where each share appears as random noise and does not reveal any information individually. The system successfully reconstructs the original image when the correct shares are combined and accurately extracts the hidden data.

The output confirms that all modules, including encoding, share generation, reconstruction, and decoding, function correctly. Additionally, quality metrics such as PSNR and SSIM indicate that image quality is well maintained. Overall, the system is reliable, secure, and efficient for data concealing applications.

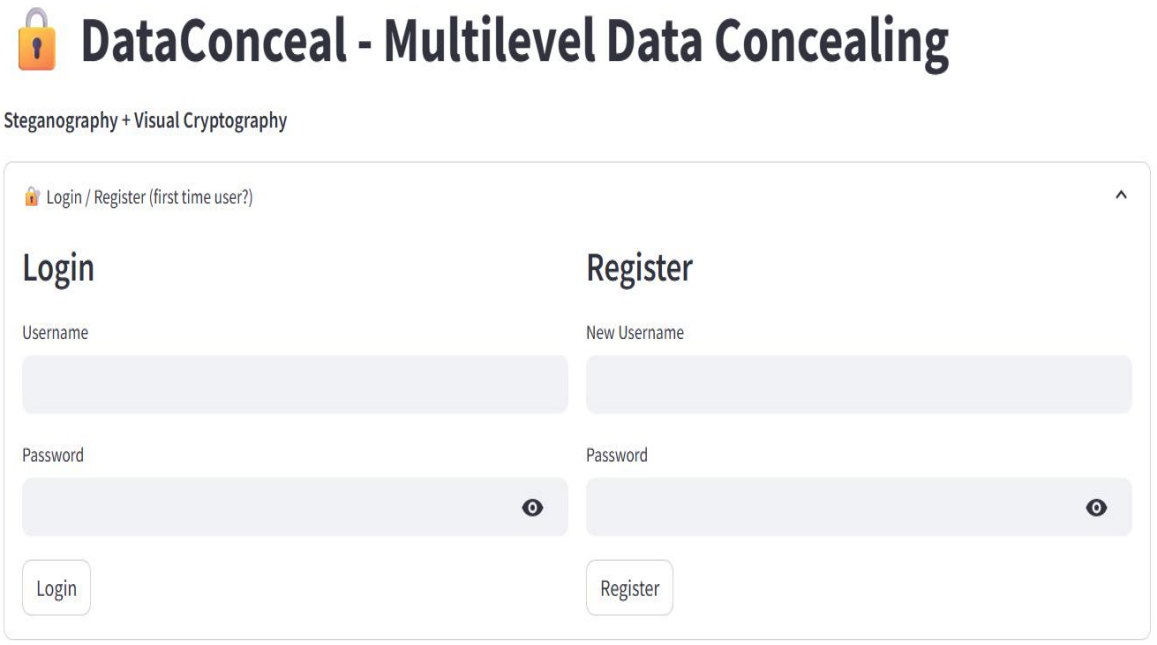

Fig 7.1:Output screen for Login and Registration

The above figure shows the login page of the Multilevel Data Concealing System using Steganography and Visual Cryptography, where users enter their credentials to securely access the application.

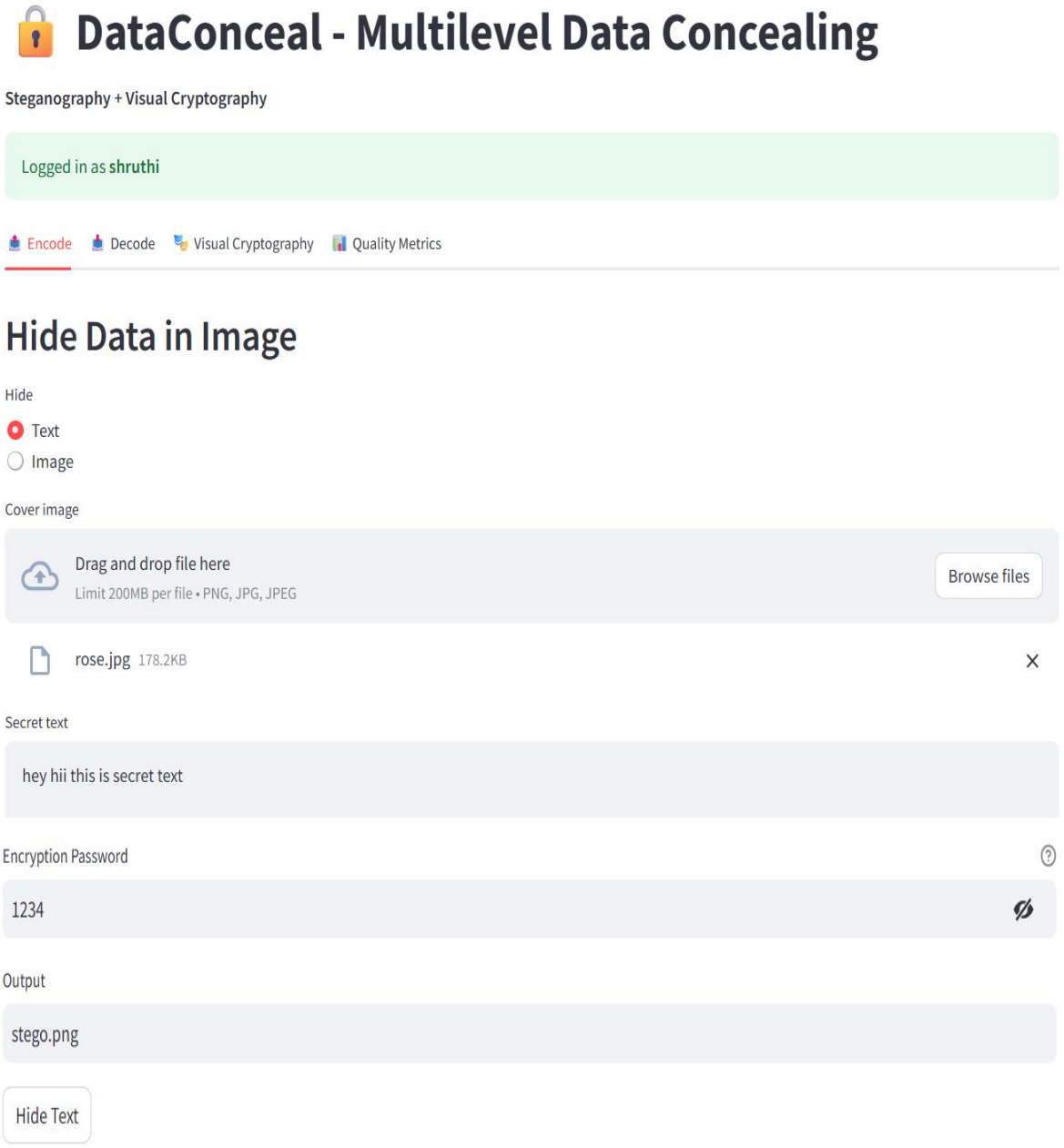

Fig 7.2: Output screen for Encoding

The above figure shows the Encode page, which allows users to hide secret data (text or image) inside a cover image using steganography. Users can upload a cover image, enter secret text or select a secret image, and optionally set a password for security. The system then generates a stego image containing the hidden data, which can be decoded later.

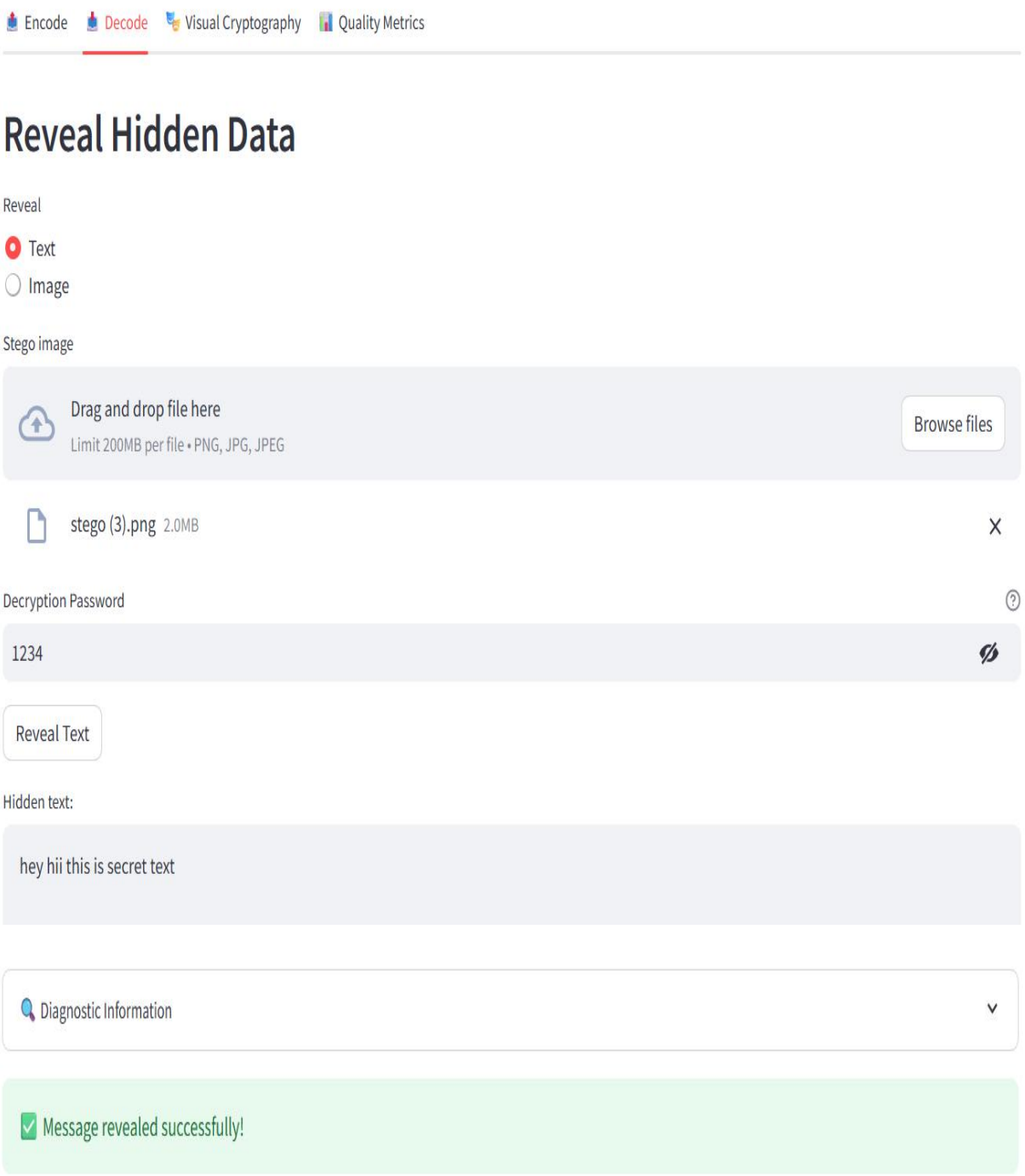

Fig 7.3: Output screen for Decoding

The above figure shows the Decode page, which allows users to extract hidden data from the stego image. Users can upload the stego image and enter the password if encryption was used. The system then processes the image and retrieves the hidden text or image, making the concealed information visible again.

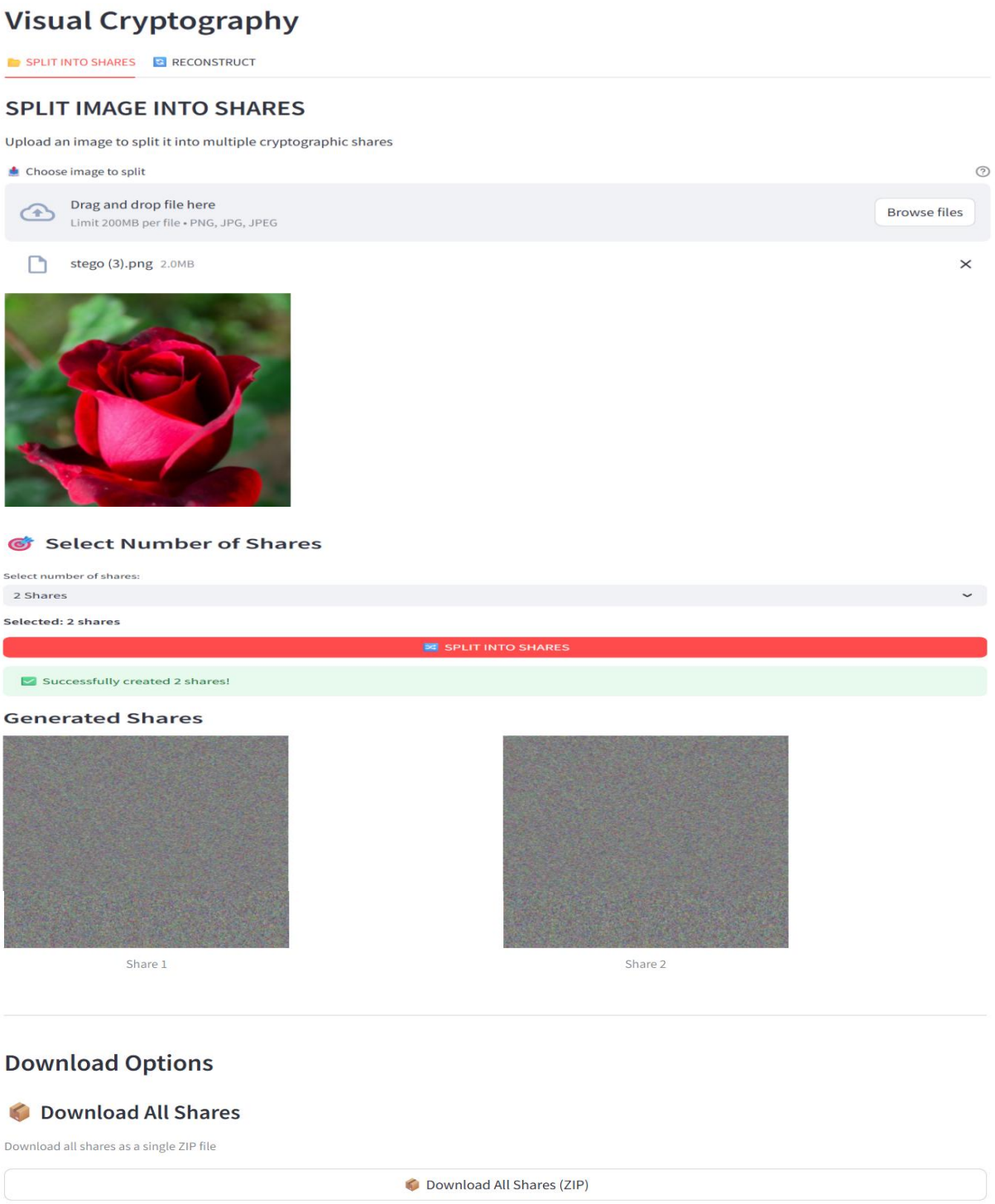

Fig 7.4: Output screen for Visual Cryptography

The above figure shows the Visual Cryptography page, which allows users to secure an image by splitting it into multiple shares. Each share appears as random noise and does not reveal any information individually. When the required shares are combined, the original image is reconstructed, providing an additional layer of security for sensitive data.

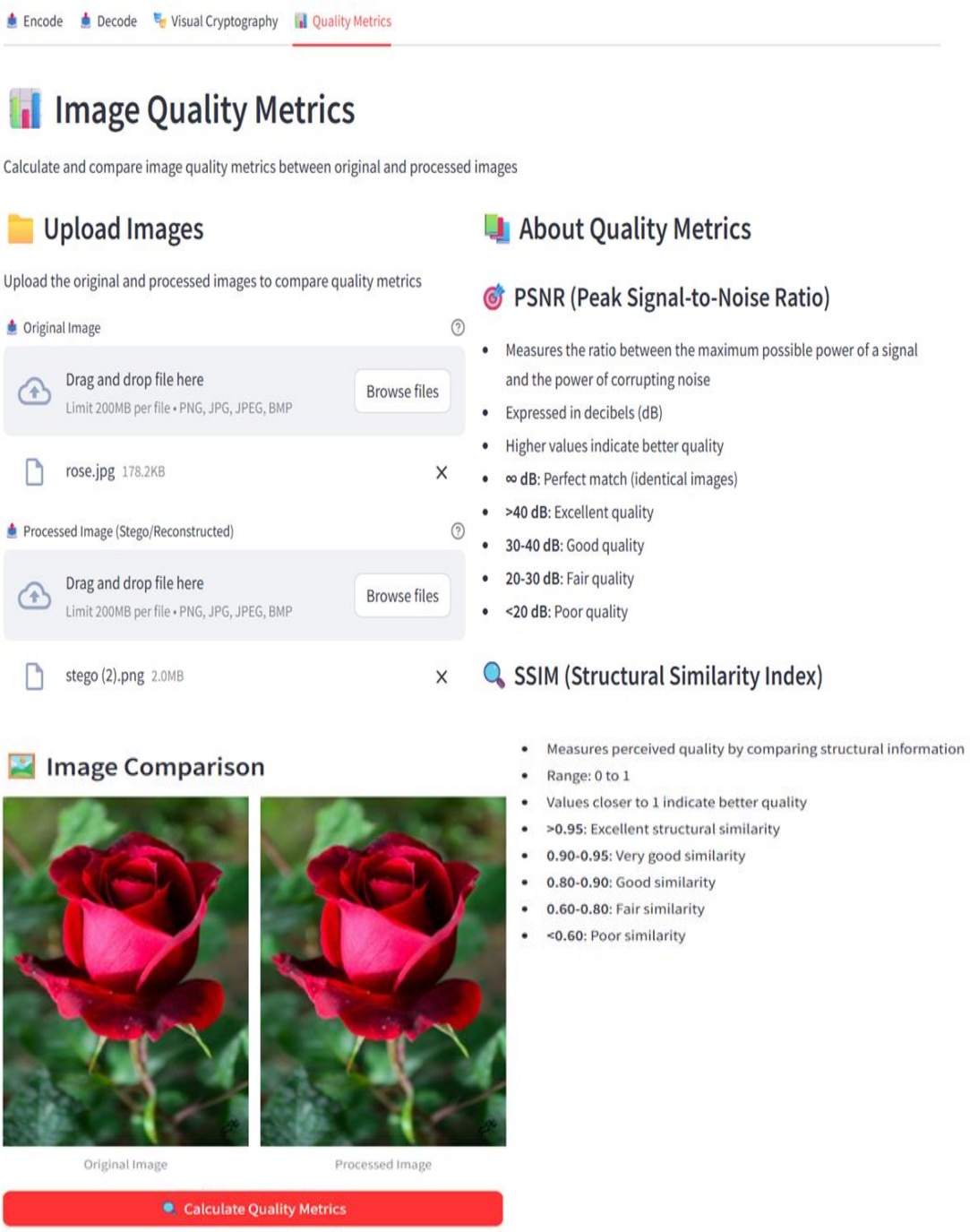

Fig 7.5: Output screen for Quality metrics

The above figure shows the Quality Metrics page, which evaluates the effectiveness of data hiding using metrics such as PSNR and SSIM. It measures how closely the stego or reconstructed image matches the original image, indicating the quality and processed image.

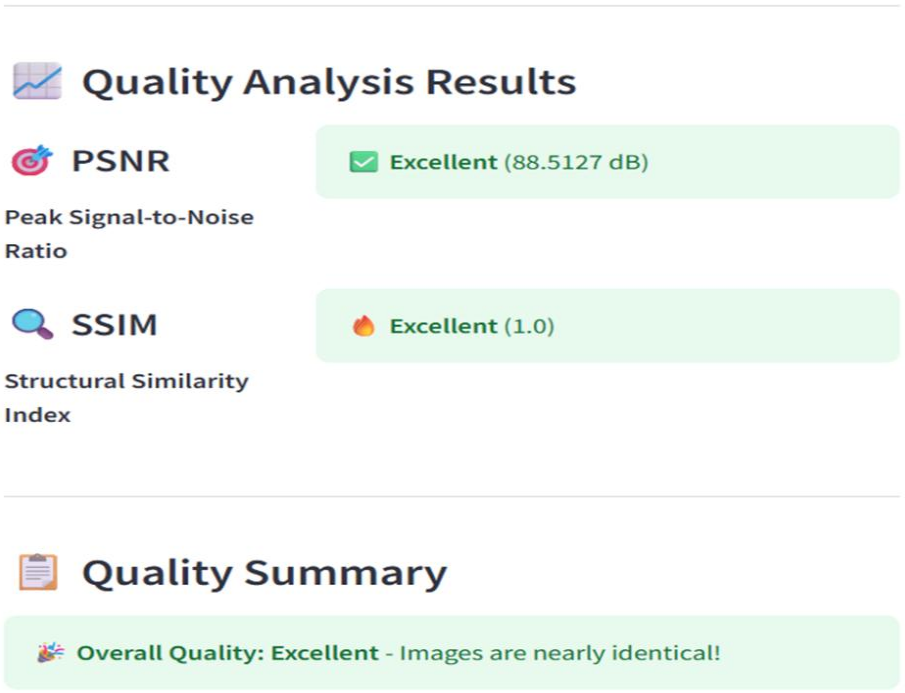

Fig 7.6: Output screen for Quality metrics result

This figure demonstrates results page displaying PSNR and SSIM values. The high PSNR (88.51 dB) and SSIM (1.0) indicate excellent image quality. The summary confirms that the original and processed images are nearly identical, ensuring minimal distortion.

8. CONCLUSION AND FUTURE ENHANCEMENT

8.1 Project Conclusion

The proposed system successfully demonstrates a secure and efficient approach for protecting sensitive information. The primary objective of the project was to enhance data security by combining two powerful techniques—steganography for hiding information within images and visual cryptography for splitting the secured data into multiple shares. The system ensures that confidential data remains hidden as well as protected from unauthorized access.

The steganography module effectively embeds secret text or images into a cover image using the Least Significant Bit (LSB) technique. This method ensures that the hidden data is invisible to the human eye while maintaining the visual quality of the image. The generated stego image does not show noticeable distortion, which makes the data concealment process highly reliable and secure.

To further strengthen security, the system incorporates visual cryptography, where the stego image is divided into multiple shares. Each share appears as random noise and does not reveal any information individually. Only when the correct shares are combined can the original stego image be reconstructed. This multi-level security mechanism ensures that even if one share is intercepted, the hidden data remains protected.

The decoding process was also successfully implemented, allowing users to reconstruct the original image from shares and extract the hidden data accurately. The system ensures that the retrieved data matches the original input, confirming its correctness and efficiency. Additionally, performance evaluation using quality metrics such as PSNR and SSIM shows that the image quality is well preserved after processing.

The system was tested using various test cases, including encoding, decoding, share generation, and reconstruction. All modules performed as expected, and the results confirmed that the system is reliable and efficient. The user-friendly interface developed using Streamlit further enhances usability, making the system accessible even to non-technical users.

Furthermore, the project highlights the importance of combining multiple security techniques to overcome the limitations of individual methods. While steganography alone hides the presence of data, visual cryptography ensures that even if the hidden data is detected, it cannot be accessed without the required shares. This layered approach significantly enhances overall system security.

In conclusion, this project provides a robust and effective solution for secure data hiding and sharing. By combining steganography and visual cryptography, it offers enhanced protection, data integrity, and confidentiality. The system can be applied in areas such as secure communication, digital watermarking, confidential data storage, and information security, making it highly valuable in real-world applications.

8.2 Future Enhancement

The proposed system can be further improved by incorporating advanced security mechanisms. One major enhancement is the integration of strong encryption algorithms such as AES or RSA before applying steganography. This would ensure that even if the hidden data is extracted, it remains encrypted and unreadable without the proper key, thereby adding an extra layer of protection.

Another important enhancement is extending the system to support multiple file formats such as audio, video, and PDF documents. Currently, the system focuses mainly on text and image data, but supporting additional formats would increase its applicability in real-world scenarios. This would allow users to securely hide and transmit different types of sensitive information.

The system can also be improved by integrating cloud storage services. This would enable users to store stego images and visual cryptography shares securely on the cloud and access them from anywhere. Cloud integration would also facilitate easy sharing of shares among authorized users, making the system more scalable and practical for distributed environments.

In addition, developing a mobile application version of the system can significantly enhance accessibility and usability. A mobile-friendly interface would allow users to perform data hiding and extraction operations directly from smartphones, making the system more convenient and widely usable.

Further improvements can be made in the visual cryptography module by implementing advanced schemes such as k-out-of-n sharing. This approach allows any k shares out of n total shares to reconstruct the original image, providing greater flexibility and security in share management. It also reduces dependency on all shares being present.

In conclusion, these future enhancements can make the system more secure, scalable, efficient, and user-friendly.

9. REFERENCES

9.1 Journals

[1] S. Kumar, R. Sharma, A. Verma, *"A Review on Image Steganography Techniques for Data Security,"* International Journal of Computer Applications, vol. 178, no. 6, pp. 20–25, 2020.

[2] P. Kaur, M. Singh, *"Multilevel Data Hiding Using Steganography and Visual Cryptography,"* International Journal of Innovative Research in Computer Science & Technology (IJIRCST), vol. 8, no. 4, pp. 10–15, 2021.

[3] A. Patel, S. Desai, *"Secure Data Transmission Using Image Steganography with Visual Cryptography,"* International Journal of Advanced Research in Computer Science, vol. 11, no. 3, pp. 45–52, 2022.

[4] R. Gupta, V. Jain, *"Hybrid Techniques for Information Hiding in Digital Images,"* International Journal of Scientific Research in Computer Science, Engineering and Information Technology, vol. 7, no. 1, pp. 30–38, 2020.

[5] M. Ahmed, S. Rehman, *"An Efficient Steganography Method Using LSB and Visual Cryptography for Secure Communication,"* Journal of Information Security and Applications, vol. 55, pp. 102–110, 2020.

[6] L. Zhao, Y. Wang, *"Visual Cryptography Combined with Steganography for Multi-Level Data Security,"* International Journal of Network Security, vol. 22, no. 5, pp. 821–830, 2020.

[7] N. Singh, P. Sharma, *"Quality Metrics Analysis in Steganography: PSNR and SSIM Approaches,"* International Journal of Computer Science and Information Technology, vol. 10, no. 2, pp. 60–66, 2021.

9.2 Books

- *Steganography in Digital Media– Principles, Algorithms, and Applications* – Jessica Fridrich

  Covers fundamental concepts and advanced techniques for hiding information in digital media.Richard Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2011.

- *Digital Image Processing* – Rafael C. Gonzalez and Richard E. Woods
  Provides core knowledge of image processing required for encoding and decoding operations.
- *Practical Cryptography in Python* – Seth James Nielson, Christopher K. Monson
  Focuses on implementing cryptographic techniques using Python.
- *Visual Cryptography and Secret Image Sharing* – Moni Naor and Adi Shamir
  Explains theoretical and practical aspects of visual cryptography.
- *Image Steganography Techniques :Methods and Tools*–Muhammad Usama and Imran Sarwar
  Discusses various steganography methods such as LSB, DCT, and hybrid approaches.
- Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Pearson Education, 2021.

9.3 Sites

- https://www.opencv.org/
- https://www.ijert.org/
- https://www.geeksforgeeks.org/multilevel-steganography-and-visual-cryptography/